REGIS UNIVERSITY

COLLEGE OF COMPUTER & INFORMATION SCIENCES

Faculty Syllabus
Fall 8 Week 2 2015

NOTE to Students: This syllabus is the SAME as the syllabus posted online, with your instructor's contact information, personal policies, grading and late policies, previous edition textbook readings, and any other modifications added in GREEN.

Course Number: CS 362
Course Title: Data Structures

INSTRUCTOR INFORMATION:

Name:	Dwayne Ockel
Telephone Number:	720.257.9876
Email:	dockel@regis.edu
Facilitator Expectations:	I make every attempt to meet the following turnaround times. Occasionally, circumstances may prevent meeting these guidelines. Forum messages: Within 24 hours Email & phone calls: Within 24-48 hours Homework & Exams: Within 1 week

Course Description

CS 362 – DATA STRUCTURES (3). Combines concepts discussed in CS361, computer programming and design, with structural programming and design. Examines data structures including arrays, structures, linked lists, queues, stacks, file organization along with file processing and algorithms used in problem solving. Emphasizes sound programming practices.

Prerequisite Courses

CS361 Control Structures

In order to successfully complete this course, students are expected to have taken the course prerequisite, CS361 (or transferred in an equivalent). From CS361, you should have a working knowledge of the following topics:

1. Practice with problem definition, solution construction and algorithmic development.
2. Practice with top-down design techniques, coding, and debugging.
3. Ability to implement code with C++ control structures used for decisions and iteration, including: if, while, do-while, for, and switch.
4. Understanding of modular code design with C++ functions, and proper parameter passing, using both pass-by-value and pass-by-reference.
5. Understanding of recursion.

Course Overview

In the pre-requisite programming course (CS361), students learned how to use various control structures (if-then, while-do, functions, etc) to control the flow of the program.

In this course, students will learn how to implement various data structures (enumerated types, arrays, records/structs, and linked lists) and how to use files (text and binary) to store and manipulate program data. Good programming practices will be encouraged throughout this course.

Ultimately, programming is a skill that requires the ability to put designs into practice. The only way to succeed is to practice this skill. Therefore, this course will require a ***significant*** amount of time each week to complete the programming assignments. The workload for the first couple of weeks will be ***at least*** as much as the workload in the last couple of weeks in CS361, and will increase further in the last half of the course. If you are not willing or able to spend the necessary time, please reconsider whether this is the correct time to attend this class.

Course Outcomes

Upon completion of this course, students should be able to:

1. Compare and contrast the data representation employed by text files and by binary files, and implement the usage of both for program data storage.
2. Define and implement Enumerated data types and Record data structures within high-level language programs.
3. Compare and contrast single and multi-dimensional arrays, and implement both within high-level language programs.
4. Compare and contrast procedural programming concepts with object-oriented programming concepts.
5. Define and use simple Classes and Objects within high-level language programs.
6. Create linked lists implemented using dynamically allocated memory within high-level language programs.

7. Differentiate between Stack and Queue data structures, and compare alternative implementations with respect to performance.
8. Choose the appropriate file and data structures for modeling a given programming problem.
9. Design, implement, test, and debug modular high-level language programs, which implement the file and data structures introduced in this course, to logically solve typical programming tasks.

Course Materials

Required Texts:

Malik, D.S. (2015). *C++ Programming: From Problem Analysis to Program Design* (7th ed.). Boston, MA: Cengage Learning; ISBN-13: 978-1285852744, ISBN-10: 1285852745

Textbook Notes:

- This is the same textbook that was used in CS361. CS362 will cover the second half.

Required Technology Tools:

1. A PC-compatible computer system running Windows.

2. A C++ Compiler -- the preferred compiler is the free **Dev-C++** compiler

The newest version of Dev-C++ (Orwell), can be downloaded from:
http://sourceforge.net/projects/orwelldevcpp/files/Setup%20Releases/

Download the most recent version:
Dev-Cpp x.x.x MinGW x.x.x Setup.exe (32-bit version)
OR
Dev-Cpp x.x.x TDM-GCC x64 x.x.x Setup.exe (64-bit version)

WARNING: If you had an older version loaded on your computer from an earlier class, such as the Bloodshed Dev-C++ compiler, ***you must uninstall it*** before installing the newer version.

Other ANSI standard C++ compilers can be used, but the facilitator will be most familiar with the Dev-C++ compiler listed above, and will most likely use the Dev-C++ compiler to grade your programs. If a student chooses to use another compiler, it is the student's responsibility to obtain any help needed to use the software, and to insure that all assigned programs compile and run correctly on the Dev-C++ compiler.

As with most of Regis learning activities, using various software applications to accomplish assignments requires students to exercise a great deal of responsibility for learning how to successfully operate the software applications. This class assumes that

you have a working knowledge of the C++ compiler from CS361. There are, however, many quality books on the market that support novice users if you need them.

Pre-Assignment

Complete the following tasks:

- Read the first week's assigned reading in the textbook (listed in the Course Assignments and Activities grid on the next page).
- Be prepared to ASK QUESTIONS on unclear areas and RESPOND to questions about information in the assigned reading.

Online Format: Sign on to worldclass.regis.edu and become familiar with the course navigation of the Web Curriculum. Complete assignments above.

Classroom-based Format: Complete assignments above by the first night of class.



Course Assignments and Activities

	Topics	Readings (7th edition of textbook)*	Activities Assignments and Associated Points**
1	Introduction to Data Structures Text Files Single Dimensional Arrays	**Various chapters:** pp. 53 – 54, 156 – 157, 203 – 204 (string review) **Chap 3**, pp. 160 – 174 (file I/O and review) **Chap 4**, pp. 217 – 219 (read input failure) **Chap 6**, pp. 423 – 432 (program example - file I/O) **Chap 8**, pp. 520 – 542 (one-dimensional arrays), pp. 576 – 588 (program examples), and pp. 557 – top of 558 (specifying filenames at execution) ***Optional:*** Odd exercises 19 – 25 on pages 179 – 180, and odd exercises 1 (a-i) – 29 on pages 591 – 598 (*answers in back of text*).	Participation in Discussions 10% for entire course Programming Assn 1 (file input and arrays) 10% of 52% NOTE: Together, all 6 programming assignments will be worth 52% of the course grade.
2	Enumerated Data Types Sort/Search using Arrays Structure Charts	**Chap 7**, pp. 466 – top of 485 (enumerated types/typedef and sample program) **Chap 8**, pp. bottom of 542 – middle of 549 (sequential search/selection sort), middle of 558 – middle of 559 (parallel arrays) **Chap 16**, pp. 1056 – middle of 1057 (sequential search), 1065 – 1068 (binary search), and 1077 – 1091 (parallel arrays program example) **Appendix F**, pp. 1328 – 1329 (character checking library review) ***Optional:*** Exercises 1 (a - h), 3, and 5 on pages 510 – 511, and exercises 1, 3, and 15 on pages 1093 – 1095 (*answers in back of text*)	Participation in Discussions
3	Records Library Functions	**Chap 7**, pp. bottom of 490 – 510 (string functions, example, and review) **Chap 9**, all (records/structs) ***Optional:*** Exercises 15 and 17 on pages 514 – 515, and odd exercises 1 - 17 on pages 641 – 645 (*answers in back of text*).	Participation in Discussions Programming Assn 2 (file I/O, enum types, parallel arrays, search/sort) 16% of 52%

4	Multi-Dimensional Arrays	**Chap 8**, pp. middle of 559 – 590 (two- and multi-dimensional arrays and review) ***Optional:*** Odd exercises 39 - 43 on pages 601 – 602 (*answers in back of text*).	Participation in Discussions Programming Assn 3 (array of records) 20% of 52% Midterm Exam 19 %
5	Pointers Structured Binary Files Dynamic Memory Allocation	**Chap 12**, pp. 812 – 841 (pointers and dynamically allocated memory) **Appendix E**, pp. middle of 1310 – 1323 (binary files)	Participation in Discussions Programming Assn 4 (2D arrays and binary files) 16% of 52%
6	Data Abstraction Classes and Objects	**Chap 10**, 650 – 698, 705 – 720 (classes, objects, data abstraction, program examples and review) **Chap 12**, pp. bottom of 841 – 852 (classes and pointers: peculiarities) **Chap 13**, 888 – middle of 952 (operator overloading and program example) ***Optional:*** Odd exercises 1 – 17 on pages 720 – 726, odd exercises 1 - 33 on pages 873 – 880, and odd exercises 1 – 11, 21 – 25 on pages 965 – 969 (*answers in back of text*).	Participation in Discussions Programming Assn 5 (classes/objects) 16% of 52%
7	Classes and Objects (templates) Linked Lists	**Chap 13,** 953 – 965 (templates and review) **Chap 17**, pp. 1058 – 1103 (linked lists) ***Optional:*** Odd exercises 29 – 37 on pages 969 – 971, and odd exercises 1 - 19 on pages 1182 - 1187 (*answers in back of text*).	Participation in Discussions
8	Stacks & Queues	**Chap 18**, pp. 1196 – 1229 (stacks) and middle of 1245 - 1262 (queues) **Quick Reviews** for chapters 3, 7, 8, 9, 10, 12 (1-27), 13, 17 (1-9) and 18 (1-6, 10-15) ***Optional:*** Odd exercises 1 – 7 on page 1282 – 1284 (*answers in back of text*).	Participation in Discussions Programming Assn 6 (linked lists) 22% of 52% Final Exam 19 %
		Total	**100%**

****Other Edition Reading Assignments:*** Alternate reading assignments for the previous edition of the textbook (6th edition) are available in the Appendix of this syllabus.

Summary of Assignments and Percentage Weight towards course grade

Assignment		Value (percent of overall course grade)
Programming Assignments (6)		
Program 1 (file I/O and arrays)	10% of 52%	
Program 2 (enum types, parallel arrays, sorting and searching)	16% of 52%	
Program 3 (array of records)	20% of 52%	
Program 4 (2D arrays and binary files)	16% of 52%	
Program 5 (classes and objects)	16% of 52%	
Program 6 (linked lists)	22% of 52%	
Total for all Programming Assignments		52%
Midterm Exam		19 %
Final Exam		19 %
Forum / Participation		10 %
Course Total		100 %

Course Policies and Procedures

Programming Assignments

Each programming assignment will involve writing a program that implements the concepts discussed in the book and class. The percentage of the grade allocated to each programming assignment is an indication of the relative effort that will be required for implementation.

Each programming assignment will involve writing two programs, using the concepts discussed in the book and class, and may also include submission of pseudocode, flowcharts, test data and/or test plans. Programming assignments will be graded according to the following criteria:

1. Source Code
   - The code is easy to read, i.e. formatted nicely (spaces/blank lines used).
   - The code follows the coding standards provided.
   - The code includes in-line documentation.
2. Coding Style
   - The program is modular.
   - The C++ programming language is used correctly (structured, non-OO).
3. Functionality
   - There are no compilation/logic/formatting errors, etc.
   - The program solves assigned problem
   - The code is efficient and elegant
4. Pseudocode/FlowChart (if required)
   – Algorithm is correctly depicted
5. Test Plan (if required)

– Includes both test input and expected output for each test case

– Includes all cases necessary to validate the program code

Note that simply because your program works does not mean you will receive full credit. The elegance and appropriateness of your solution, in addition to your adherence with the program requirements/specifications will be considered. It is important to learn proper programming practices, not simply use brute force methods to accomplish a task. As a result, some aspects of the grading are subjective and at the discretion of the instructor.

Furthermore, it is impossible for the grading rubrics to contain/describe every possible scenario in which points could be deducted. The rubric is intended to serve as a general guideline for grouping deducted points into appropriate categories. However, there may be deductions against a category that are not specifically described in the rubric itself. When points are deducted, I try to subtract points from the most appropriate category. If you have any questions about why points were deducted that you feel are not adequately explained in the narrative provided at the end of the rubric, you are encouraged to ask questions. Just keep in mind that the rubric is not "all inclusive" in terms of identifying every possible scenario for which points may be deducted, especially when the deductions are for issues of elegance, efficiency, convention, etc., as mentioned above. It is also important to remember that the requirements for the programming assignments serve an academic purpose. You may be required to do something a certain way simply because it demonstrates a concept or methodology that is important to learn. That does not mean it is necessarily the best or only way to accomplish the task. However, you are still expected to implement your programs in accordance with the given requirements/specifications.

Also, keep in mind that point deduction(s) reflect the severity (or potential severity) of the error(s). Just because a mistake may seem minor or trivial (such as failing to initialize a variable), the consequences of such an error on program execution could be tremendous. Thus, don't expect that the amount of a deduction will be based purely on the "size" or perceived "smallness" of the oversight/error. It is important to address issues like this early, as the rubrics and penalties will get increasingly more involved/severe as the course and program moves on. In subsequent courses, for example, errors that may be accepted in this class with a small deduction will be automatic zeros.

With regards to submission, as soon as you submit your assignment to the dropbox, it becomes eligible for grading. This is true even if you submit your assignment early. Once your assignment is graded, the grading is final, you cannot submit another version (even if it is before the deadline). Thus, make sure that you take the time to make sure that the version you submit to the dropbox is ready for grading. Further, both parts of the assignment (program 1 and program 2) must be submitted together.

Late Assignment Policy for CS362 Programming Assignments

Late programming assignments will be graded and then 2% will be deducted for each day the assignment is late, ***up to 5 days late***. No programming assignment will be accepted more than ***5 days*** after the official due date. Therefore, any programming assignment turned in more than ***5 days*** late will be given a grade of **zero**, and no feedback will be given.

I do not accept reworked/second chance assignments. All grading is final and will be done against initial submissions of all coursework, including forum posts/assignments.

Exams

There will be a midterm exam and a final exam. Exam questions will be cumulative, taken from reading assignments and course content. Exams will **not** be accepted late.

A Note about Online Exams

Online Exams are automatically graded. However, the automatic grader does not always correctly grade short answer questions due to spacing, punctuation, etc. If you believe a question has been incorrectly scored, please let me know and I will adjust your score as warranted.

Also, remember that the exams run during a window of time and just because you have finished an exam, doesn't mean everyone else has. Do NOT post any questions/information about the content of the exams until after the exam window has expired. Doing so can give an unfair advantage to those who have not yet taken the exam and is considered a form of academic dishonesty. Do not share any information about the content of the exams with any other student in the class, inside or outside the class. If you are contacted by anyone soliciting information about an exam, please let your instructor know immediately.

Additionally, using exam information from previous instances of the course (whether it be exams you took or someone else took) is prohibited. Any evidence that this has occurred will be result in failing the course and referral to the department for further disciplinary action. If you have taken any of the online exams previously, you should be contacted regarding taking alternative exams. If you have previously taken an online exam in this course and have not received information on how to obtain and submit an alternative exam, please contact the instructor for further instructions. You will not receive credit for taking the same online exam more than once.

Adding this course during the Drop/Add Period

If you added this course during the drop/add period, after class began on Monday, you are responsible for ***immediately*** notifying the instructor that you joined the course late. None of the course due dates will be extended for you. Even if a due date already passed when you added the course, late points will still be deducted.

Repeating the course

If you are repeating this course (due to a previous withdraw or low grade), you are responsible for ***immediately*** notifying the instructor. Course assignments that you submitted when you last took the course cannot be repeated -- you will be required to complete alternate assignments.

Plagiarism

Plagiarism includes submitting code or anything else that was obtained from any other person, publication, or any internet web source. ***All work submitted in CS362 must be your own.***

In cases of suspected cheating or plagiarism, the instructor will discuss the matter with the student(s) involved. The instructor reserves the right to question any student orally or in writing about any assignment, and to use the evaluation of the student's understanding of the assignment and of the submitted solution as evidence of cheating. All cheating incidents will be reported to the Computer Science department, and may also be reported to the Academic Integrity Board for further action.

I keep an extensive database of previously submitted programs and will compare submissions against this database. Using a friend's code who has previously taken the course is not a good idea.

Further, do not consult with other students in the class about the programming assignments, unless expressely permitted to do so for a group project. Do not post any messages to the forum containing information about how you solved or intend to solve any part of a programming assignment, or ask any questions that might cause someone to reveal such information, until after the due date plus the late submission period has ended. Any such questions should be directed to the instructor privately via email.

Participation

Class participation/effort is important because we can all learn from each other. Your participation points can make a difference in the final grade. Participation means:

1. a. Present in class every session (classroom)
   b. Present in the forum every week (online)
2. a. Effectively responds to questions from the facilitator (classroom)
   b. Regularly checks forum and posts all required items by the deadlines (online)
3. Interacts/replies to other students in classroom/forum discussions.

Weekly Discussion Points Distribution for Participation

9 points per correct Discussion Question/Example response each week
(9 pts ***total*** for ***both*** questions in week 8)
7 points for ***at least one*** question or comment about the week's topics

(each week in weeks 1 – 7)
7 points per Weekly Feedback in weeks 1 – 7
12 points for Course Feedback in week 8

Discussion Question/Example responses will be graded using the rubric below.
Partial points may be given for late, incomplete, or partially incorrect posts.
No points will be given for items posted ***more than one week*** past the due date.

Week 1 there are 32 possible points
Week 2 there are 32 possible points
Week 3 there are 23 possible points
Week 4 there are 23 possible points
Week 5 there are 23 possible points
Week 6 there are 23 possible points
Week 7 there are 23 possible points
Week 8 there are 21 possible points

Total forum points available: **200 pts** = 10% of total course grade

CS362 Discussion Question Rubric

Demonstration of Subject Matter Comprehension	The initial post ***correctly addressed all parts*** of the discussion question.	There were ***minor*** mistakes in initial post, but the student ***corrected them all*** by the end of the week.	There were ***major*** mistakes in the initial post, but the student ***corrected them all*** by the end of the week.	The initial post contained only ***minor uncorrected*** mistakes, OR there were ***major*** mistakes in the initial post and the student ***only corrected some*** of them by the end of the week.	There were ***major*** mistakes in the initial post, and the student ***did not correct any*** of them by the end of the week.	Student did not post an answer to the discussion question.
Subject Points	9	8	7	6 - 4	3 - 1	0
Timeliness	Initial post by due date.	Initial answer posted 1 - 7 days late (10% deducted per day late)				No post within 7 days of due date
Late Deductions	0	–10% to –70%				–100% (no credit)

Participation/Forum postings must be substantive and relevent. Simple posts that state agreement or "good job", etc. should be avoided. Students are responsible for reading all existing postings before posting to avoid repetitive posts which can clutter the forum and distract from the class.

CC&IS Grading Scale

Letter Grade	Percentage	Grade Point
A	93 to 100	4.00
A–	90 to less than 93	3.67
B+	87 to less than 90	3.33
B	83 to less than 87	3.00
B–	80 to less than 83	2.67
C+	77 to less than 80	2.33
C	73 to less than 77	2.00
C–	70 to less than 73	1.67
D+	67 to less than 70	1.33
D	63 to less than 67	1.00
D-	60 to less than 63	.67
F	Less than 60	0

Additional information about grading can be found in the latest edition of the University Catalog, available at http://www.regis.edu/Academics/Course%20Catalog.aspx.

Errors/Misprints, etc.

I reserve the right to make any necessary corrections throughout the course as the result of unintentional errors, misprints, broken links, etc. You are responsible for reading all forum posts throughout the course. They may contain information about corrections, changes, etc. to the content of this syllabus or the course.

Extra Credit

There is no extra credit available for this course.

CC&IS Policies and Procedures

Each of the following CC&IS Policies & Procedures is incorporated here by reference. Students are expected to review this information each term, and agree to the policies and procedures as identified here and specified in the latest edition of the University Catalog, available at http://www.regis.edu/Academics/Course%20Catalog.aspx or at the link provided.

- The CC&IS Academic Integrity Policy.
- The Student Honor Code and Student Standards of Conduct.
- Incomplete Grade Policy, Pass / No Pass Grades, Grade Reports.
- The Information Privacy policy and FERPA. For more information regarding FERPA, visit the U.S. Department of Education.
- The HIPPA policies for protected health information. The complete Regis University HIPAA Privacy & Security policy can be found here: http://www.regis.edu/About-Regis-University/University-Offices-and-Services/Auxiliary-Business/HIPAA.aspx.
- The Human Subjects Institutional Review Board (IRB) procedures. More information about the IRB and its processes can be found here: http://regis.edu/Academics/Academic-Grants/Proposals/Regis-Information/IRB.aspx.

The CC&IS Policies & Procedures Syllabus Addendum summarizes additional important policies including, Diversity, Equal Access, Disability Services, and Attendance & Participation that apply to every course offered by the College of Computer & Information Sciences at Regis University.

A copy of the CC&IS Policies & Procedures Syllabus Addendum can be found here: https://in2.regis.edu/sites/ccis/policies/Repository/CCIS%20Syllabus%20Addendum.docx.

Incompletes

I very rarely issue incomplete grades and typically do so only when documented evidence of an extraordinary circumstance is presented (such as a serious illness or death in the family). Additionally, an incomplete will only be considered if you are currently passing the course at the time of the request and have completed at least half of the course work (based on available points from all applicable forum participation, quizzes/exams, papers, homework assignments, etc.) Furthermore, it may not be possible to achieve full credit on the work that must be made up. Incompletes will not be given due to time management issues, employment assignments/obligations, falling behind, etc.

APPENDIX

Alternate Reading Assignments for 6th Edition of the Textbook

Week	Learning Topics	*Alternate Readings* ***(6th edition)***
1	Introduction to Data Structures Text Files Single Dimensional Arrays	**Various chapters:** pp. 53-54, 153-154, 188-189 (string review) **Chap 3**, pp. 157 – 171 (file I/O and review) **Chap 4**, pp. bottom of 214 – 217 (read input failure) **Chap 6**, pp. middle of 408 – 418 (program example - file I/O) **Chap 8**, pp. 506 – 527 (one-dimensional arrays), pp. 559 – 571 (program examples), and pp. top of 541- top of 542 (specifying filenames at execution) ***Optional:*** Odd exercises 17 - 23 on pages 175 - 176, and odd exercises 1 (a-k) - 25 on pages 573 - 580 (*answers in back of text*).
2	Enumerated Data Types Sort/Search using Arrays Structure Charts	**Chap 7**, pp. 452 – top of 471 (enumerated types/typedef) **Chap 8**, pp. bottom 527 – 534 (selection sort), middle of 542 – middle of 543 (parallel arrays) **Chap 16**, pp. 1016 – middle of 1017 (sequential search), 1025 – 1028 (binary search), and 1034 – 1048 (parallel arrays program example) **Appendix F**, pp. 1282 – 1283 (character checking library review) ***Optional:*** Exercises 1 (a - h), 3, and 5 on pages 496 – 498, and exercises 1, 3, and 13 on pages 1050 – 1051 (*answers in back of text*)
3	Records Library Functions	**Chap 7**, pp. bottom of 476 – 496 (string functions, example, and review) **Chap 9**, all (records/structs) ***Optional:*** Exercise 15 and 17 on page 500, and odd exercises 1 - 13 on pages 622 - 625 (*answers in back of text*).
4	Multi-Dimensional Arrays	**Chap 8**, pp. middle of 543 – 573 (two- and multi-dimensional arrays) ***Optional:*** Odd exercises 37 - 41 on pages 583 - 584 (*answers in back of text*).
5	Pointers Structured Binary Files Dynamic Memory Allocation	**Chap 12**, pp. 782 – 808 (pointers and dynamically allocated memory) **Appendix E**, pp. middle of 1264 – 1277 (binary files)
6	Data Abstraction Classes and Objects	**Chap 10**, 630 – middle of 673, middle of 679 – 694 (classes, data abstraction, program examples and review) **Chap 12**, pp. 809 – 819 (pointers and class peculiarities) **Chap 13**, 854 – 915 (operator overloading) ***Optional:*** Odd exercises 1 – 15 on pages 695 – 699, odd exercises 1 - 27 on pages 840 – 846, and odd exercises 1 – 11, 17 – 23 on pages 928 – 932 (*answers in back of text*).

7	Classes and Objects Linked Lists	**Chap 13,** 915 – 928 (templates and review) **Chap 17**, pp. 1058 – 1103 (linked lists) ***Optional:*** Odd exercises 25 – 31 on pages 932 – 934, and odd exercises 1 - 17 on pages 1138 - 1142 (*answers in back of text*).
8	Stacks & Queues	**Chap 18**, pp. 1150 - 1183 (stacks) and 1199 - 1216 (queues) **Quick Reviews** for chapters 3, 7, 8, 9, 10, 12 (1-27), 13, 17 (1-9) and 18 (1-6, 10-15) ***Optional:*** Odd exercises 1 – 5 on page 1238 (*answers in back of text*).

